

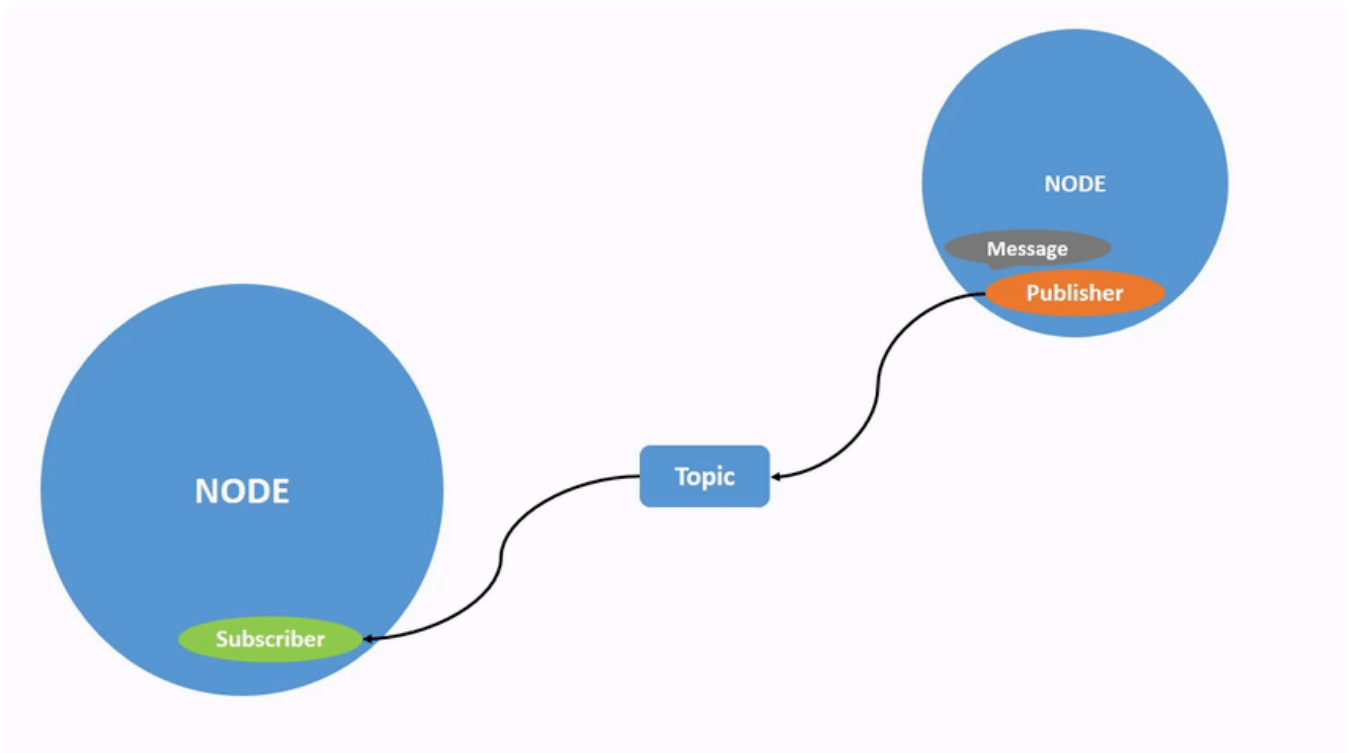
7. ROS2 Topic Communication

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. Topic Communication Overview

Topic communication is the most frequently used communication method in ROS2. A publisher publishes data on a specified topic, and subscribers who have subscribed to that topic can receive the data.

Topic communication is based on the publish/subscribe model, as shown in the figure:



The characteristics of topic data transmission are: from one node to another. The object sending data is called the **publisher**, and the object receiving data is called the **subscriber**. Each topic needs a name, and the transmitted data also needs a fixed data type.

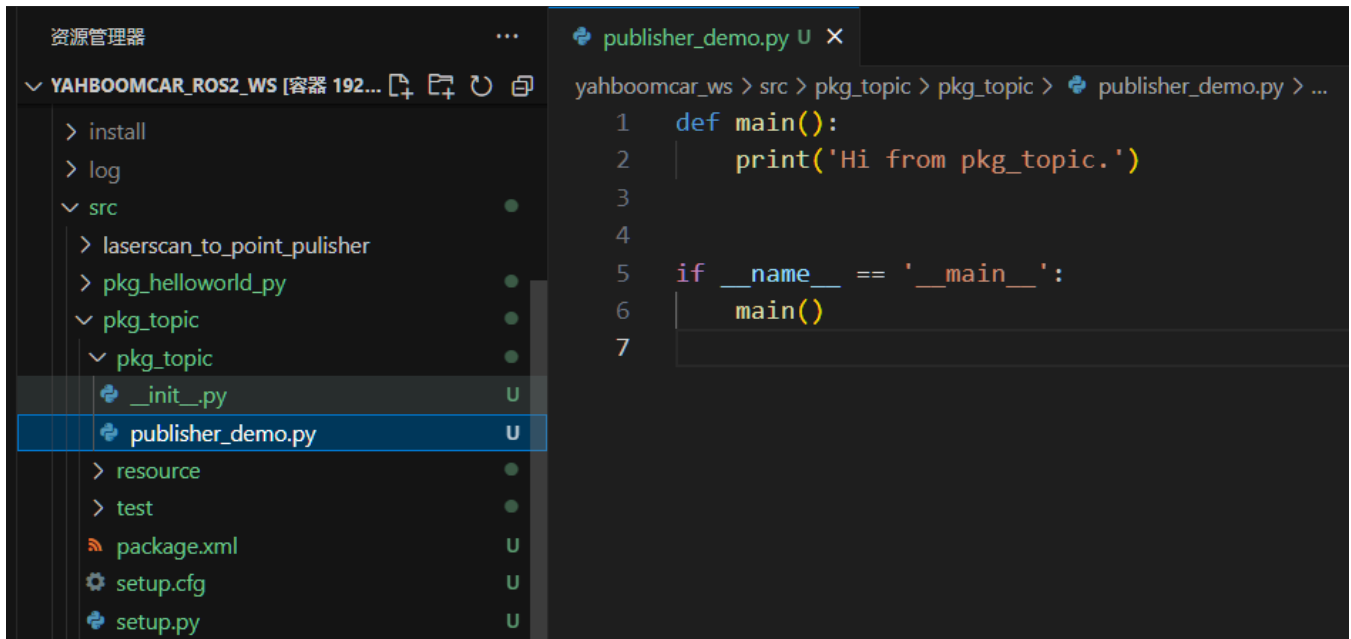
Next, we will explain how to implement topic communication between nodes using the Python language.

2. Create New Package

- Navigate to the workspace src directory:
- Create the pkg_topic package:

```
ros2 pkg create pkg_topic --build-type ament_python --dependencies rclpy --node-name publisher_demo
```

After executing the above command, the pkg_topic package will be created, along with a publisher_demo node, and the related configuration files will already be set up.



3. Publisher Implementation

3.1 Create Publisher

Next, edit [publisher_demo.py] to implement the publisher functionality. Add the following code:

```
# Import rclpy library
import rclpy
from rclpy.node import Node

# Import String message
from std_msgs.msg import String

# Create a Topic_Pub node subclass inheriting from Node base class with a parameter name
class Topic_Pub(Node):
    def __init__(self, name):
        super().__init__(name)
        # Create a publisher using create_publisher function with parameters:
        # topic data type, topic name, message queue length
        self.pub = self.create_publisher(String, "/topic_demo", 1)
        # Create a timer that enters the interrupt handler every 1 second with parameters:
        # interrupt handler execution interval, interrupt handler function
        self.timer = self.create_timer(1, self.pub_msg)
    # Define interrupt handler function
    def pub_msg(self):
        msg = String() # Create a String type variable msg
        msg.data = "Hi,I send a message." # Assign value to msg.data
        self.pub.publish(msg) # Publish topic data

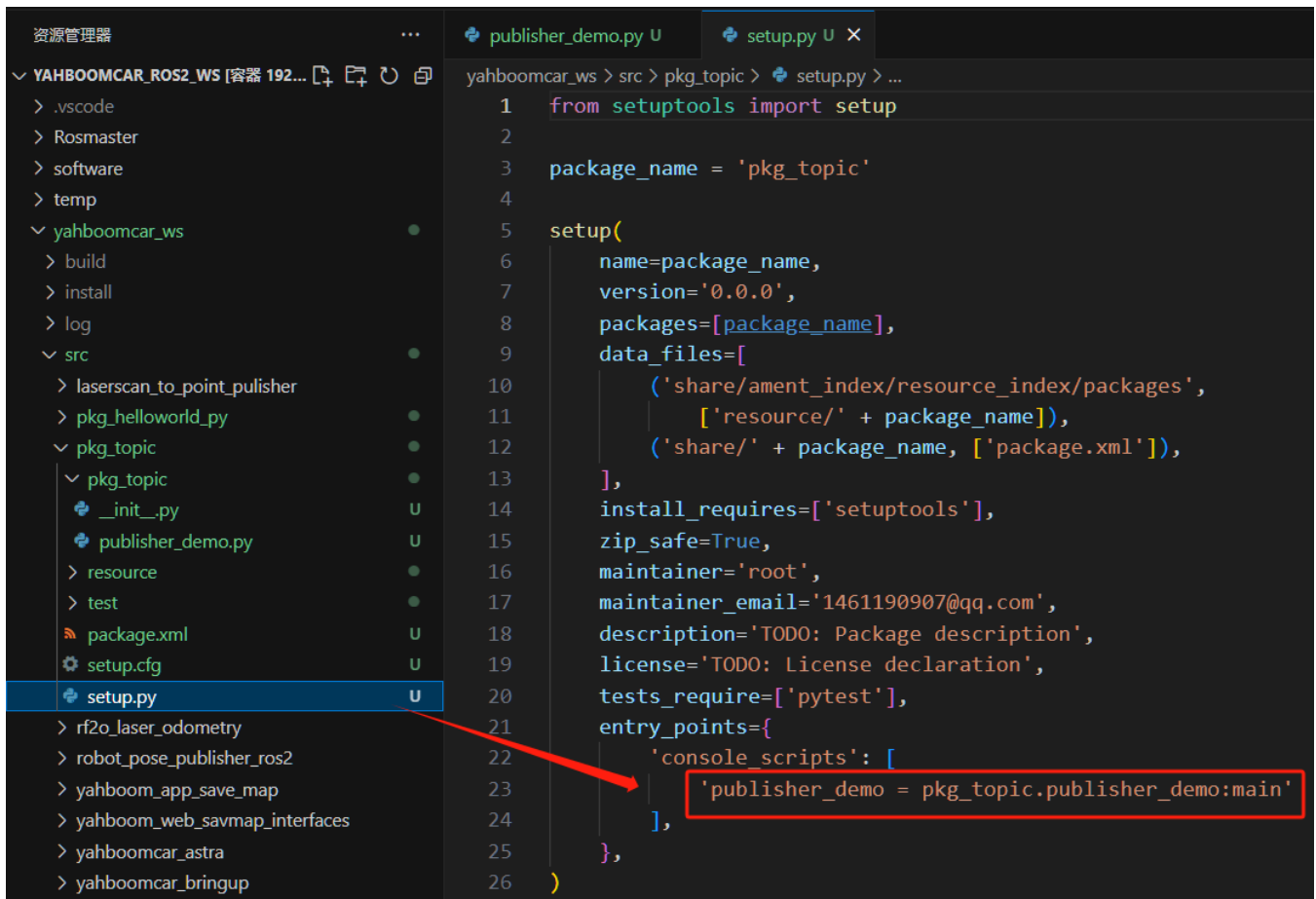
# Main function
def main():
    rclpy.init() # Initialize
    pub_demo = Topic_Pub("publisher_node") # Create Topic_Pub class object with node name as
parameter
```

```

rclpy.spin(pub_demo)      # Execute rclpy.spin function with Topic_Pub class object as
parameter
pub_demo.destroy_node()    # Destroy node object
rclpy.shutdown()           # Shutdown ROS2 Python interface

```

3.2 Edit Configuration File

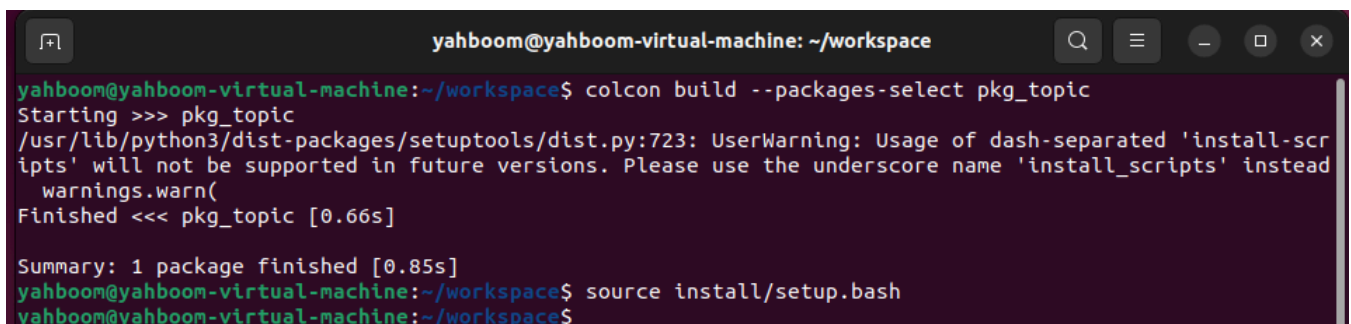


3.3 Build Package

- Build the package:

```
colcon build --packages-select pkg_topic
```

- Refresh environment variables in the workspace:



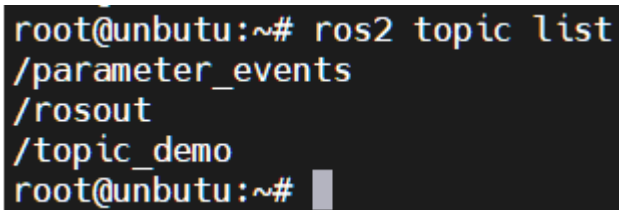
3.4 Run Program

- After refreshing environment variables, run the command:

```
ros2 run pkg_topic publisher_demo
```

When the program runs successfully, there is no output. We can use the `ros2 topic` tool to view data. First, check if there are topics published. Open another terminal and enter:

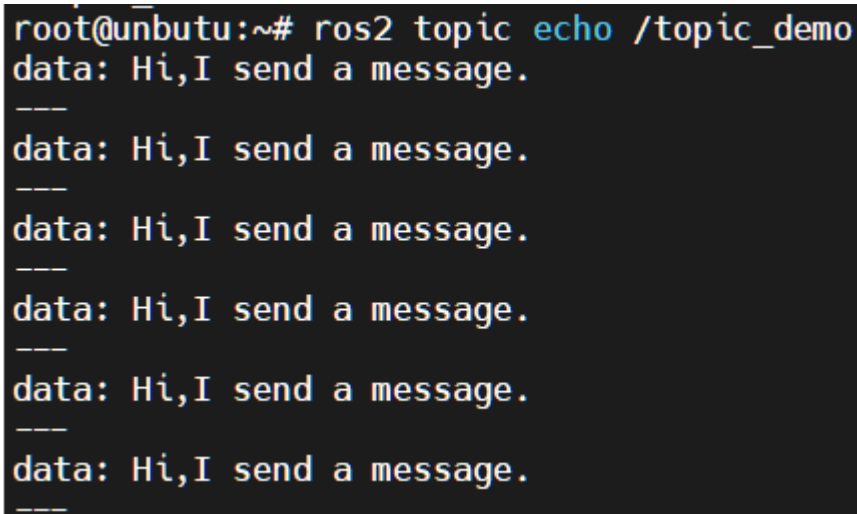
```
ros2 topic list
```



```
root@unbutu:~# ros2 topic list
/parameter_events
/rosout
/topic_demo
root@unbutu:~#
```

`topic_demo` is the topic data defined in the program. Next, we use `ros2 topic echo` to print this data. Enter in the terminal:

```
ros2 topic echo /topic_demo
```



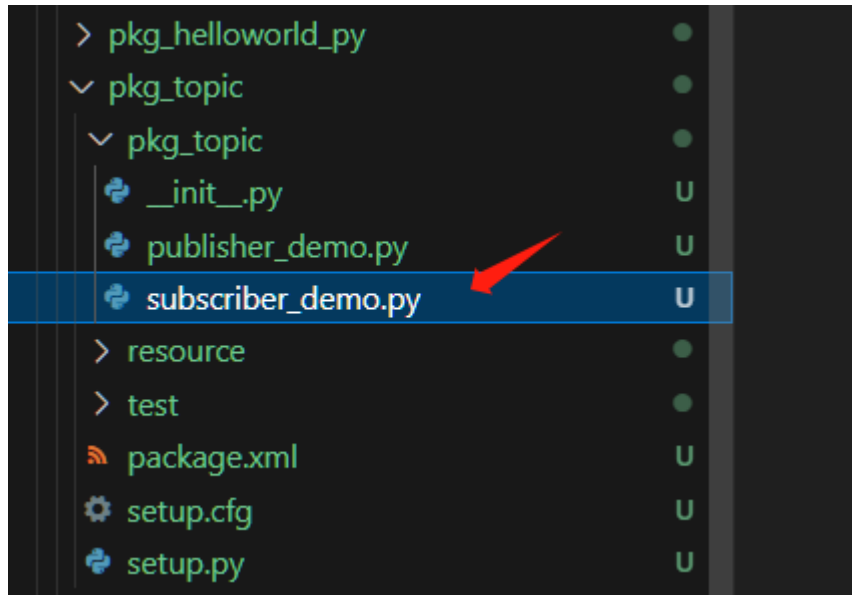
```
root@unbutu:~# ros2 topic echo /topic_demo
data: Hi,I send a message.
---
data: Hi,I send a message.
---
data: Hi,I send a message.
---
data: Hi,I send a message.
---
data: Hi,I send a message.
---
data: Hi,I send a message.
---
```

As you can see, the terminal prints "Hi,I send a message." which matches `msg.data = "Hi,I send a message."` in the code.

4. Subscriber Implementation

4.1 Create Subscriber

Create a new file [subscriber_demo.py] in the same directory as [publisher_demo.py]:



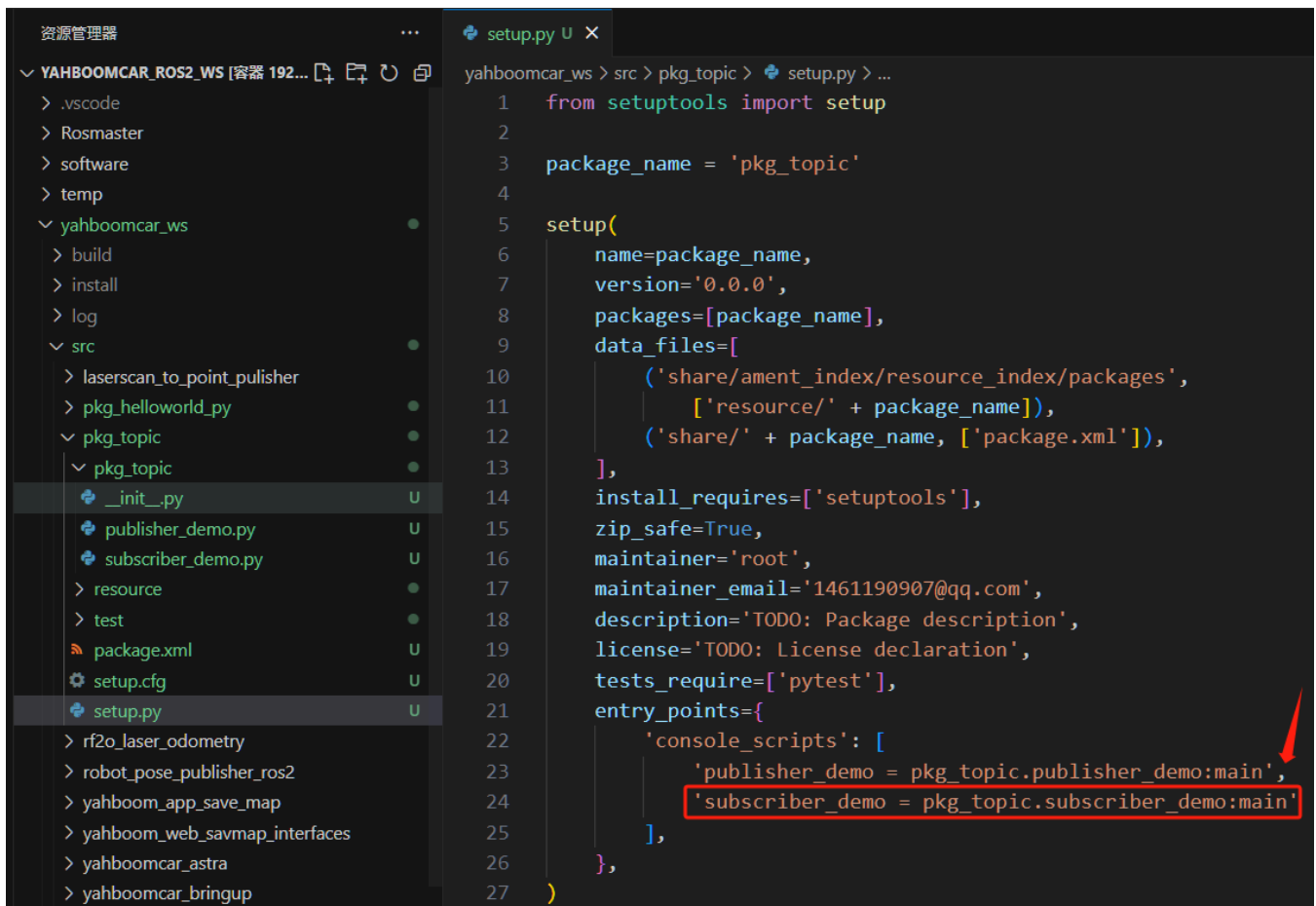
Next, edit [subscriber_demo.py] to implement the subscriber functionality. Add the following code:

```
# Import related libraries
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Topic_Sub(Node):
    def __init__(self, name):
        super().__init__(name)
        # Create subscriber using create_subscription with parameters:
        # topic data type, topic name, callback function name, queue length
        self.sub = self.create_subscription(String, "/topic_demo", self.sub_callback, 1)
    # Callback function executes: print received information
    def sub_callback(self, msg):
        # print(msg.data, flush=True)
        self.get_logger().info(msg.data)

def main():
    rclpy.init() # ROS2 Python interface initialization
    sub_demo = Topic_Sub("subscriber_node") # Create object and initialize
    rclpy.spin(sub_demo)
    sub_demo.destroy_node() # Destroy node object
    rclpy.shutdown() # Shutdown ROS2 Python interface
```

4.2 Edit Configuration File



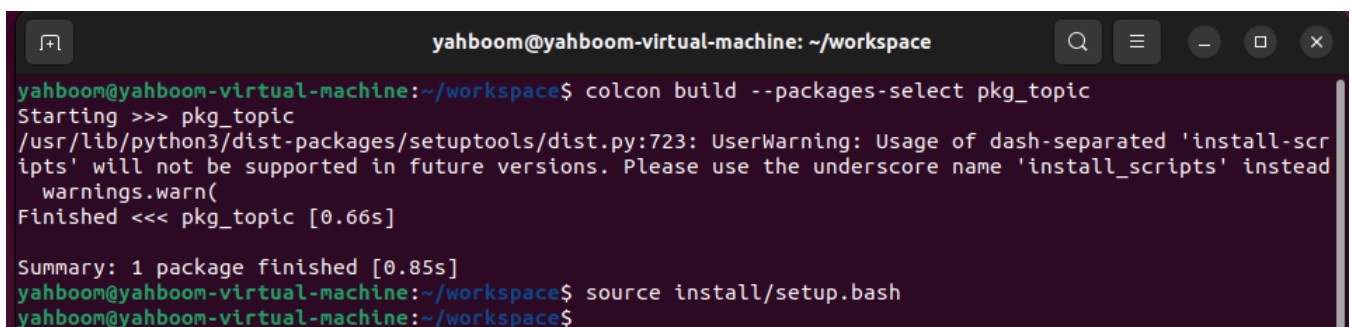
```
1  from setuptools import setup
2
3  package_name = 'pkg_topic'
4
5  setup(
6      name=package_name,
7      version='0.0.0',
8      packages=[package_name],
9      data_files=[
10         ('share/ament_index/resource_index/packages',
11          ['resource/' + package_name]),
12         ('share/' + package_name, ['package.xml']),
13     ],
14     install_requires=['setuptools'],
15     zip_safe=True,
16     maintainer='root',
17     maintainer_email='1461190907@qq.com',
18     description='TODO: Package description',
19     license='TODO: License declaration',
20     tests_require=['pytest'],
21     entry_points={
22         'console_scripts': [
23             'publisher_demo = pkg_topic.publisher_demo:main',
24             'subscriber_demo = pkg_topic.subscriber_demo:main'
25         ],
26     },
27 )
```

4.3 Build Workspace

- Build the package:

```
colcon build --packages-select pkg_topic
```

- Refresh environment variables in the workspace:



```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select pkg_topic
Starting >>> pkg_topic
/usr/lib/python3/dist-packages/setuptools/dist.py:723: UserWarning: Usage of dash-separated 'install-scripts' will not be supported in future versions. Please use the underscore name 'install_scripts' instead
  warnings.warn(
Finished <<< pkg_topic [0.66s]

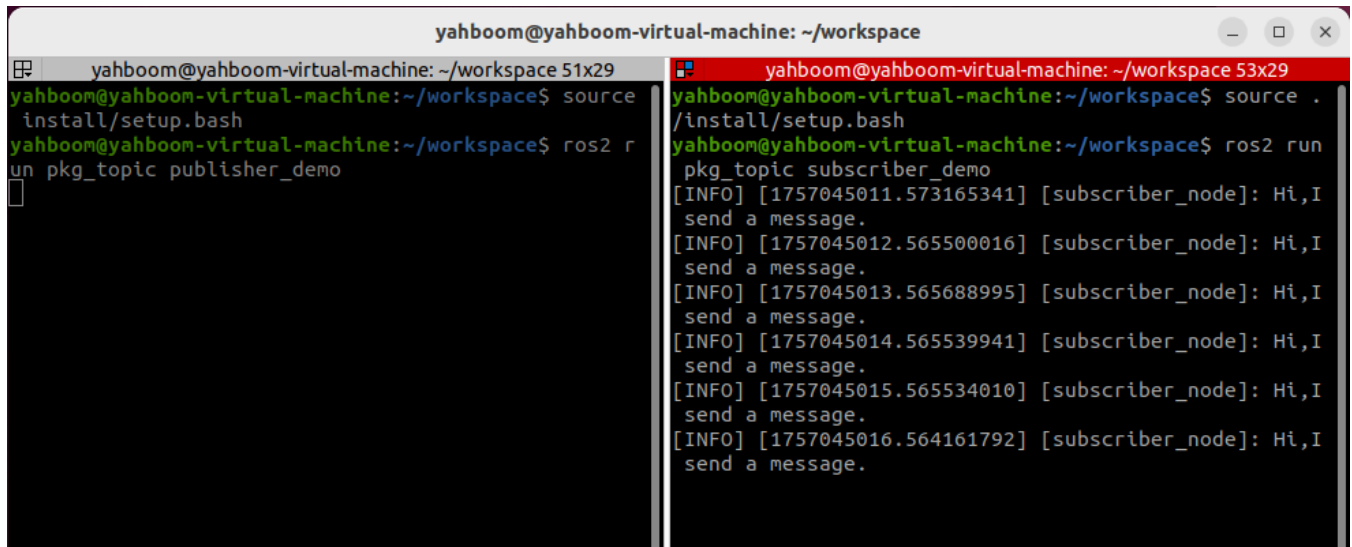
Summary: 1 package finished [0.85s]
yahboom@yahboom-virtual-machine:~/workspace$ source install/setup.bash
yahboom@yahboom-virtual-machine:~/workspace$
```

4.4 Run Program

Split terminal and execute:

```
# Start publisher node
ros2 run pkg_topic publisher_demo

# Start subscriber node
ros2 run pkg_topic subscriber_demo
```



```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 51x29
yahboom@yahboom-virtual-machine:~/workspace$ source
install/setup.bash
yahboom@yahboom-virtual-machine:~/workspace$ ros2 r
un pkg_topic publisher_demo

yahboom@yahboom-virtual-machine: ~/workspace 53x29
yahboom@yahboom-virtual-machine:~/workspace$ source .
/install/setup.bash
yahboom@yahboom-virtual-machine:~/workspace$ ros2 run
pkg_topic subscriber_demo
[INFO] [1757045011.573165341] [subscriber_node]: Hi,I
send a message.
[INFO] [1757045012.565500016] [subscriber_node]: Hi,I
send a message.
[INFO] [1757045013.565688995] [subscriber_node]: Hi,I
send a message.
[INFO] [1757045014.565539941] [subscriber_node]: Hi,I
send a message.
[INFO] [1757045015.565534010] [subscriber_node]: Hi,I
send a message.
[INFO] [1757045016.564161792] [subscriber_node]: Hi,I
send a message.
```

As shown in the image above, the terminal running the subscriber will print the information published by the publisher on /topic_demo.